

Chapter 4

Lexical Analysis in Practice

NFA and DFA scanning, language quirks, hand-written lexers and flex

Compiler Construction Module

Based on Lecture 4 (“Part 2: Lexical analysis”), expanded

How to read this chapter

Chapter 2 said what a token is, and Chapter 3 built the machine that recognises one. This lecture looks at the scanner as a working program. It covers how the lexer serves the parser, how to run an NFA without building a DFA, how many token automata become one, what to do when nothing matches, which languages break the model, and how to write a scanner in C or generate one with `flex`. Material already in Chapter 3 is only summarised here. Every `flex` run and trace printed in this chapter is real output. The last section maps each problem of Problem Set 4 to the part of the chapter that prepares it.

1 What a Lexer Is For

The parser wants to think in terms like “a while-statement is the keyword `while`, a parenthesis, an expression...”. It should never see a space or a comment. The **lexer** (or **scanner**) absorbs all of that.

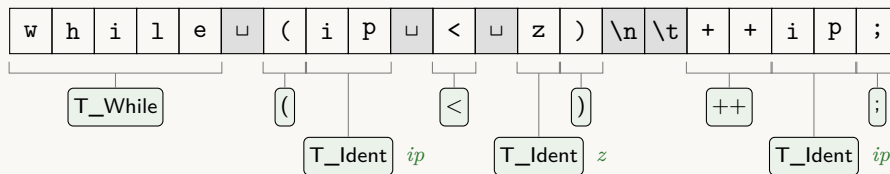
Mechanism: The four jobs of a lexer

1. **Cut** the character stream into *lexemes*.
2. **Label** each lexeme with a *token*, which is what the parser receives.
3. **Record** identifiers (and often numbers) in the *symbol table*.
4. **Discard** blanks, newlines and comments.

The parser drives: it calls the lexer each time it needs one more token (`getNextToken()` in the textbook, `yylex()` in `flex`).

Worked example: From characters to tokens

For the lexer, `while (ip < z)` followed by a newline, a tab and `++ip;` is one flat stream. The grey cells are discarded:



Three things happened. The blanks vanished. Only **T_Ident** carries data (*which* identifier), while a parenthesis needs none. And the lexer decided where each token ends: it did not stop at **whi**, and it did not split **++**. That last decision is the main problem of this chapter (Section 4).

Why a separate lexer? The slide gives three reasons.

- **Simplicity.** Without a lexer, every grammar rule would have to allow blanks and comments between any two symbols. With one, the rule is just *whileStmt* $\rightarrow$ **WHILE** (*expr*) *stmt*. Each half also gets the right tool: finite automata for tokens, context-free grammars for structure.
- **Efficiency.** The lexer is the only phase that touches every character, so it deserves specialised techniques such as DFA tables and block-buffered input.
- **Portability.** Only the scanner deals with encodings, line endings and files.

Concept: Token, pattern, lexeme

- A **token** is a pair $\langle \text{name}, \text{attribute} \rangle$. The name is a terminal of the grammar; the attribute carries extra data. Examples: $\langle \text{Ident}, \text{ip} \rangle$, $\langle \text{Operator}, < \rangle$, $\langle \text{"("}, \text{NIL} \rangle$.
- A **pattern** describes the set of strings that form lexemes of a token, for example “a letter followed by letters and digits”.
- A **lexeme** is an actual piece of the source matching a pattern, such as *ip*, *<* or *)*.

Token	Pattern	Lexemes	Attribute
WHILE	while	while	none
IDENT	letter (letter digit)*	ip , count1	pointer to symbol-table entry
RELOP	< <= = <> > >=	<, >=	which operator
NUMBER	digits, optional fraction and exponent	60, 3.14E-2	the value

Token names are invented by the compiler writer, as **#define** constants or an **enum** shared by lexer and parser. **T_Ident**, **IDENT** and **ID** are the same idea spelled differently.

Defining a lexer takes three steps: define the tokens, give each one a pattern, and give an algorithm that cuts the input. The first two depend on the language. The third is always the same, which is why tools can generate it.

Choosing tokens. The usual classes are one token per keyword, one per

punctuation symbol, one for identifiers, several for operators, and one or more for constants. The rule for splitting or merging is this: *give two lexemes different token names exactly when the grammar must treat them differently*. All six relational operators sit in the same grammar position, so they can share one token RELOP with an attribute. `+` and `*` must be separate tokens, because the grammar encodes their different precedence.

Attributes and the symbol table. Identifiers carry a pointer to their symbol-table entry and constants carry their value. The lexer inserts each new name once, so every occurrence of `x` gets the *same* pointer, and later phases compare pointers instead of strings.

2 Patterns: Regular Expressions Revisited

A pattern is a set of strings, which is a language, and token patterns are written as regular expressions. Quick reminder (slides 56–57; Chapter 2 has the details):

- An **alphabet** Σ is a finite set of characters, such as $\{a, b\}$. A **string** is a finite sequence of them, such as `aabba`. A **language** is a set of strings, such as $\{a, b, abab, babba\}$.
- The **regular expressions** are built from a character a (language $\{a\}$) and ε (language $\{\varepsilon\}$) using three operations: R_1R_2 (concatenation), $R_1 \mid R_2$ (union of the languages) and R^* (Kleene closure: zero or more repetitions). (R) only groups. Star binds tightest, then concatenation, then $\mid$. The **regular languages** are exactly those described this way.

Worked example: Reading a regular expression: even numbers (slide 57)

$$(+ \mid - \mid \varepsilon) (0 \mid 1 \mid \dots \mid 9)^* (0 \mid 2 \mid 4 \mid 6 \mid 8)$$

Read it as three concatenated parts: an optional sign, then any digits, then one even digit. To test a string, try to assign each character to a part.

- `-24` matches (sign `-`, middle `2`, last `4`), and so does `0` (empty middle).
- `0034` matches too, because leading zeros are allowed.
- `7` fails (odd last digit), and so does `+` (no final digit).

Shorthands add no power (slide 58):

- one or more: $r^+ = rr^*$;
- zero or one: $r? = r \mid \varepsilon$;
- character classes: $[abc] = a \mid b \mid c$ and $[a-z] = a \mid \dots \mid z$;
- **regular definitions**, which name patterns: $letter \rightarrow A \mid \dots \mid Z \mid a \mid \dots \mid z$, $digit \rightarrow 0 \mid \dots \mid 9$, $id \rightarrow letter (letter \mid digit)^*$.

A name may only refer to *earlier* names, so definitions can always be expanded away. A recursive definition such as $expr \rightarrow (expr) \mid id$ is not regular, and that is where the parser takes over.

Worked example: The slide's token patterns

- **Identifiers** $[a-zA-Z_][a-zA-Z_0-9]^*$. Because the first character is never a digit, a scanner can tell a name from a number by its first character alone.
- **Integers** $[+-]?[0-9]^+$.
- **Floats**: an optional sign, a mantissa and an optional exponent:
 $[+-]?((([0-9]+(\.[0-9]*)?)|\.[0-9]+)([eE][+-]?[0-9]+)?)$
 The mantissa is either digits with an optional point and fraction (3, 3., 3.14) or a point and digits (.5). It accepts 3.14e-2 and 1E10, and rejects ., e5 and 1e. It also accepts plain integers like 17, so floats and integers *overlap*. Rule priority must decide (Section 5).
- **Strings** $"([a-zA-Z0-9]|\[a-zA-Z])^*" is a teaching simplification, since it allows no spaces. A realistic version is $"([\^"\\\n]|\\.)*"$: any character except a quote, a backslash or a newline, or a backslash followed by anything.$

Warning / Gotcha: A sign inside the number pattern is a trap

With rules ID, INT = $[+-]?[0-9]^+$ and MINUS, the input x-1 gives $\langle \text{ID}, x \rangle \langle \text{INT}, -1 \rangle$, because -1 is longer than -. The parser then sees two operands in a row. Real languages keep the sign out of literals and treat it as unary minus in the grammar.

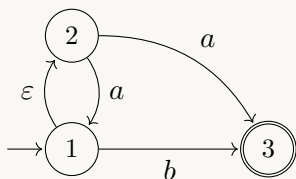
3 Running an NFA Directly

There are two ways to execute patterns. Method 1 converts them to an NFA and simulates it. Method 2 converts the NFA to a DFA and runs that. Chapter 3 did method 2. Method 1 is worth knowing on its own, because a DFA is nothing but a precomputed table of NFA-simulation steps.

3.1 The NFA and Thompson's construction

An NFA is a five-tuple $M = (Q, \Sigma, \Delta, s_0, F)$ whose transitions form a *relation* $\Delta \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$. A transition-table entry is therefore a *set* of states, possibly empty. A string is accepted if *some* path spells it.

Worked example: The slide's NFA

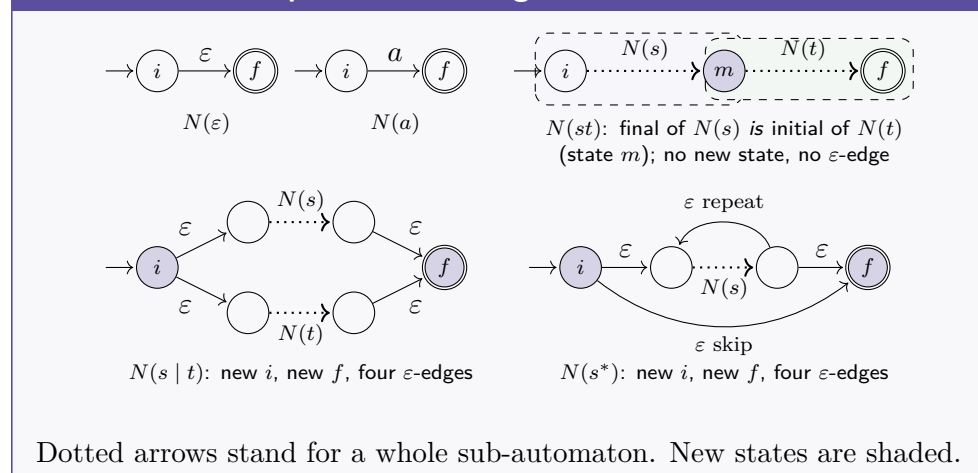


State	a	b	ϵ
1	$\emptyset$	$\{3\}$	$\{2\}$
2	$\{1, 3\}$	$\emptyset$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

Each trip around $1 \xrightarrow{\epsilon} 2 \xrightarrow{a} 1$ reads one a . The machine exits by $1 \xrightarrow{b} 3$ or $2 \xrightarrow{a} 3$, so $L = L(a^*b \mid a^+)$.

The lecture's Thompson construction comes from the Dragon book. It differs from Chapter 3's in one place: for concatenation it **merges** the final state of $N(s)$ with the initial state of $N(t)$, so there is no new state and no ε -edge. Problem Set 4 uses this convention.

Mechanism: Thompson's rules, merged-concatenation version



Proposition 3.1 (Properties of Thompson NFAs, slide 63) $N(r)$ has at most twice as many states as r has operators and operands.

- (P2) It has one initial state with no incoming edge, and one accepting state with no outgoing edge.
- (P3) Every non-accepting state has either one outgoing edge, or two outgoing edges that are both ε .

Proof sketch. (P2) and (P3) hold for the base cases and are preserved by every rule. Union and star add a fresh initial state with two ε -edges and a fresh final state with none. For concatenation, the merged state m had no outgoing edges (as the final state of $N(s)$) and no incoming edges (as the initial state of $N(t)$). So after the merge it has exactly $N(t)$'s outgoing edges. (P1) follows from the exact count below. $\square$

Proposition 3.2 (Exact counts, merged convention). If r has n_{leaf} leaves (symbols or ε), $n_{|}$ alternations, n_* stars and n concatenations, then

$$|Q| = 2(n_{leaf} + n_{|} + n_*) - n = 2(\#operators + \#operands) - 3n.,$$

$$|E| = n_{leaf} + 4(n_{|} + n_*).$$

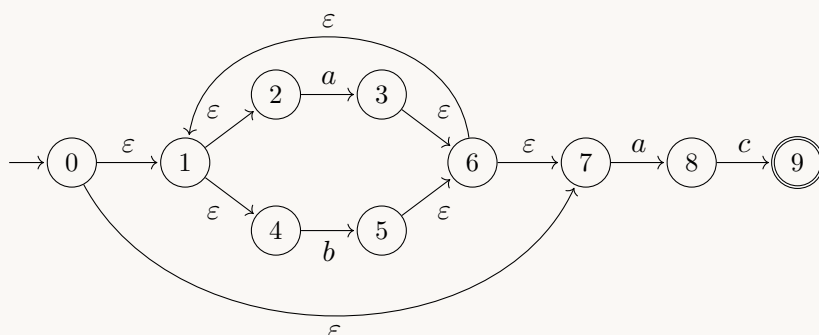
The bound (P1) is tight exactly when there is no concatenation.

Proof. A leaf creates 2 states and 1 edge. Union and star each create 2 states and 4 edges. A concatenation merges two states into one, removing one state and adding no edge. $\square$

Key insight. (P2) is what makes merging safe. Build a^* and b^* "lazily" as single states that are both initial and accepting with a loop, then merge them for a^*b^* . You get one state with loops on a and b , which wrongly accepts ba .

Worked example: Thompson for $(a \mid b)^*ac$

Parse first: $((a \mid b)^* \cdot a) \cdot c$, with 4 leaves, 1 alternation, 1 star and 2 concatenations. So $|Q| = 2 \cdot 6 - 2 = 10$ and $|E| = 4 + 8 = 12$.



States 7 and 8 are the merge points of the two concatenations. States 0, 1 and 6 have two ϵ -edges, and every other non-final state has one edge, as (P3) says. The 8-state NFA on slide 63 is *not* built by these rules: its initial state has an incoming edge.

3.2 Simulating an NFA

Do not choose a path. Keep the *set* of all states the NFA could be in.

- $move(S, c)$ is the set of states reachable from S by one c -edge.
- ϵ -closure(T) is the set of states reachable from T by zero or more ϵ -edges. It is a plain graph search, and each state is pushed at most once.

Mechanism: NFA simulation (slide 64)

```

1  $S \leftarrow \epsilon$ -closure( $\{s_0\}$ );  $c \leftarrow nextChar()$ ;
2 while  $c \neq eof$  do
3    $S \leftarrow \epsilon$ -closure( $move(S, c)$ );  $c \leftarrow nextChar()$ ;
4 if  $S \cap F \neq \emptyset$  then return “yes”;
5 else return “no”;

```

After reading a prefix w , S is exactly the set of states reachable by a path spelling w (induction on $|w|$).

Worked example: Simulating the NFA above on bac

Read	$move$	S after closure
(start)		$\{0, 1, 2, 4, 7\}$
b	$\{5\}$	$\{1, 2, 4, 5, 6, 7\}$
a	$\{3, 8\}$	$\{1, 2, 3, 4, 6, 7, 8\}$
c	$\{9\}$	$\{9\}$, which contains the final state: accept

One character can pull in a cascade: after **b**, the closure goes $5 \rightarrow 6 \rightarrow 7$ and $6 \rightarrow 1 \rightarrow 2, 4$.

Proposition 3.3 (Cost, slide 73). *On an input of length $|S|$:*

- an arbitrary NFA costs $O(|S| |Q|^2)$;
- a Thompson NFA costs $O(|S| |Q|)$;
- a DFA costs $O(|S|)$.

Proof. One step scans the outgoing edges of the states it visits. With a Boolean array for membership, each state is visited at most once per step. In an arbitrary NFA, $|Q|$ states may each have $|Q|$ successors, so a step scans $\Theta(|Q|^2)$ edges. **The square comes from the number of edges.** By (P3), a Thompson state has at most 2 outgoing edges, so a step costs $O(|Q|)$. A DFA step is a single table lookup. $\square$

For a pattern of length m , NFA simulation costs $O(m)$ memory and $O(|S| m)$ time. A DFA costs $O(|S|)$ time but may need up to 2^m states. A compiler's token rules are fixed and its inputs are long, so lexers build the DFA once.

4 Many Tokens at Once

A simulated NFA answers “is this whole string in the language?”. A lexer has many patterns and an unsegmented stream, and it must decide where the next token ends and which token it is. The slides offer two ways (slide 65):

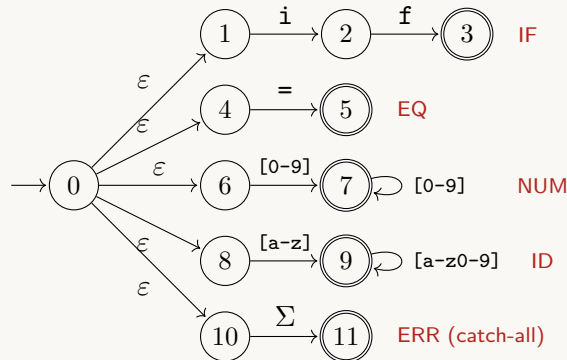
- simulate each token's NFA separately from the current position, in turn or in parallel;
- **merge** all the NFAs into one: a new start state gets an ε -edge to each token NFA, and each accepting state is labelled with its token name.

In practice the second way is used (slide 71): one simulation does all the work, and it is the automaton we later turn into a DFA.

Worked example: Four tokens, one merged NFA

IF = if, EQ = '=', NUM = $[0-9]^+$ and ID = $[a-z][a-z0-9]^*$, plus a catch-all rule ERR on states 10 and 11, which is explained below.

Key insight. NFA simulation is subset construction done lazily: one character at a time, and only for the sets the input actually visits.



On $x = 60$, with blanks skipped, the lexer outputs $\langle \text{ID}, x \rangle \langle \text{EQ} \rangle \langle \text{NUM}, 60 \rangle$ (slide 66). Each token here ends where no automaton can continue, so there is nothing to decide yet. The next input is harder.

Ambiguity 1: where does a token end? $\text{ifu26} = 60$ could be cut as IF, ID(u26), ... or ID(ifu26), ... or ID(ifu), NUM(26), ... The rule is the **principle of the longest matching prefix** (*maximal munch*): at each position, take the longest prefix that matches *any* token, then restart after it. This makes ifu26 one identifier and $\leq$ one operator.

Mechanism: Implementing longest match

Run the merged NFA from the current position and remember the **last match**: the label and end position from the last time S contained an accepting state. When S becomes empty or the input ends, emit the last match and restart right after it. Every character read after the match is **pushed back**, because it belongs to the next token.

Worked example: Tracing $\text{ifu26}=60$

The input is written without blanks, so the slide's remark that “= is read and re-inserted” appears exactly. Priority order: IF, EQ, NUM, ID, ERR.

Read	S	Accepting labels	Last match / action
i	{2, 9, 11}	ID, ERR $\Rightarrow$ ID	(ID, 1)
f	{3, 9}	IF, ID $\Rightarrow$ IF	(IF, 2)
u, 2, 6	{9}	ID	(ID, 5)
=	$\emptyset$	stuck	emit ID ifu26 , push back =
=	{5, 11}	EQ	(EQ, 1)
6	$\emptyset$	stuck	emit EQ , push back 6
6, 0	{7}	NUM	(NUM, 2); end of input: emit NUM 60

That is ten reads for eight characters. Look at the row for f : IF is the best label after two characters, but the scanner keeps reading, and the longer ID match replaces it.

Ambiguity 2: one lexeme, two tokens. `if` matches both `IF` and `ID` with the same length. The specification either forbids such overlaps or sets a **priority** (keywords before identifiers). When S contains several accepting states, the highest-priority label wins. In `flex`, priority is simply rule order. An alternative is to have one identifier rule and look the lexeme up in a keyword table, which is what the C scanner of Section 7 does.

Ambiguity 3: nothing matches. Add a **catch-all** rule $10 \xrightarrow{\Sigma} 11$ with the lowest priority, which reports an error and skips one character. It can never steal a real token. Any other rule that matches 2 or more characters wins on length, and any other rule that matches exactly 1 character wins the tie on priority.

Summary: Lexical analysis with an NFA (slide 72)

Build an NFA per token, merge them under a new start state with labelled accepting states, scan while tracking the last match, break ties by priority, and catch the rest with a catch-all rule.

5 Scanning with a DFA

A DFA has no ε -edges and exactly one transition per (state, letter) pair, so $\delta : Q \times \Sigma \rightarrow Q$ is a function. Simulating it (slide 77) needs no sets at all:

Mechanism: DFA simulation

```

1  $s \leftarrow s_0; \quad c \leftarrow nextChar();$ 
2 while  $c \neq eof$  do  $s \leftarrow \delta(s, c); \quad c \leftarrow nextChar();$ 
3 if  $s \in F$  then return “yes”;
4 else return “no”;

```

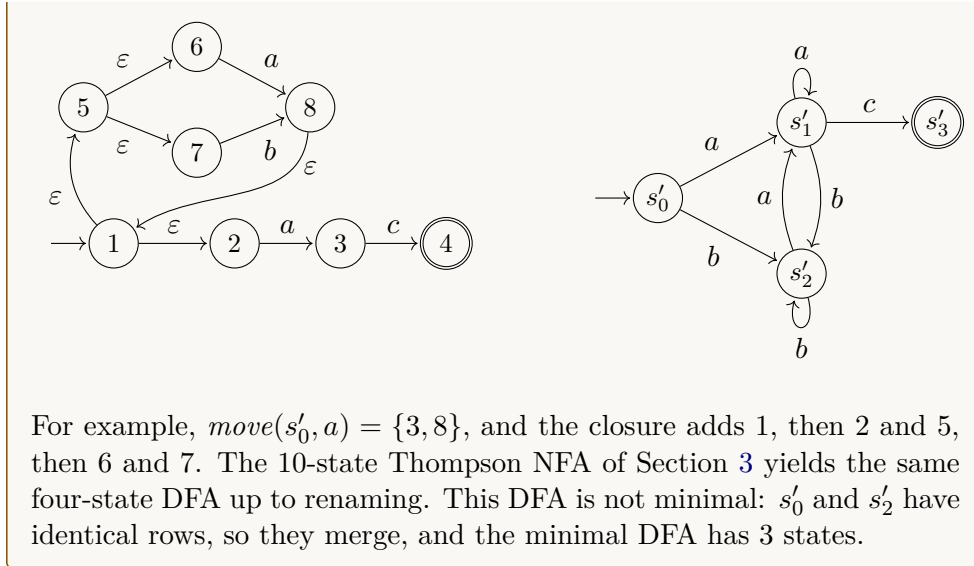
One table lookup per character: $O(|S|)$, whatever the number of states.

The **subset construction** (Chapter 3) precomputes the NFA simulation for every possible input:

- each DFA state is a set of NFA states, and the first is $\varepsilon\text{-closure}(s_0)$;
- $\delta(S, c) = \varepsilon\text{-closure}(\text{move}(S, c))$, the same step as in the simulation;
- a set is accepting if it contains an accepting NFA state.

Worked example: Slide 76: $(a \mid b)^*ac$ from Mogensen's 8-state NFA

DFA state	NFA set	a	b	c
s'_0	$\varepsilon\text{-cl}(1) = \{1, 2, 5, 6, 7\}$	s'_1	s'_2	—
s'_1	$\{1, 2, 3, 5, 6, 7, 8\}$	s'_1	s'_2	s'_3
s'_2	$\{1, 2, 5, 6, 7, 8\}$	s'_1	s'_2	—
s'_3 (accepting)	$\{4\}$	—	—	—



5.1 The combined DFA for several tokens

A DFA state that contains accepting NFA states of several tokens takes the label of the highest-priority one (slide 78). Scanning then works as with the NFA. The lecture's example (slides 79–80) has the tokens IF, ID `[a-zA-Z_][a-zA-Z_0-9]*`, NUM `[+-]?[0-9]+` and FLOAT (the float pattern of Section 2), in that priority order. Figure 1 shows the merged NFA and Figure 2 the resulting DFA; the table lists the subset construction.

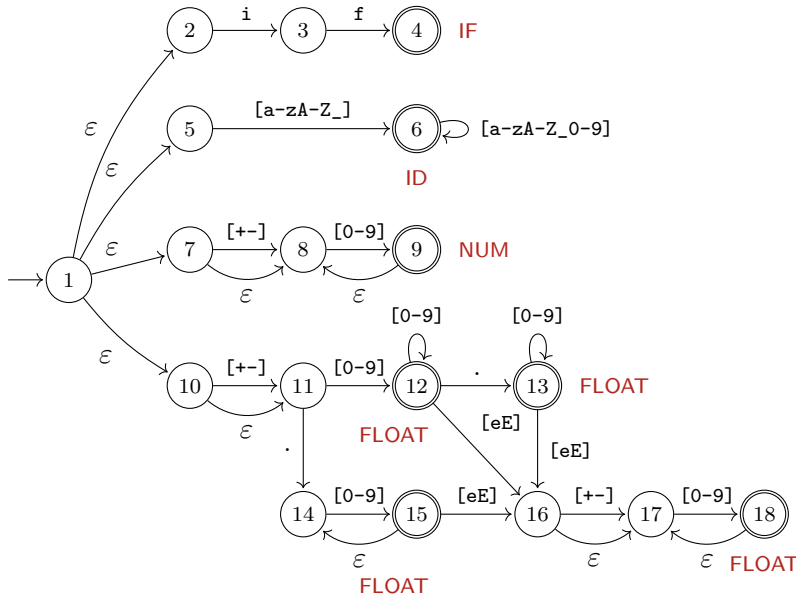


Figure 1: The combined NFA (slide 79). State 12 accepts because FLOAT also matches plain integers.

DFA	NFA set	Label	Transitions
A	{1, 2, 5, 7, 8, 10, 11}	—	i→B, other letters→D, [+~]→F, [0-9]→G, .→E
B	{3, 6}	ID	f→C, other letters/digits→D
C	{4, 6}	IF (over ID)	letters/digits→D
D	{6}	ID	letters/digits→D
E	{14}	—	[0-9]→H
F	{8, 11}	—	[0-9]→G, .→E
G	{8, 9, 12}	NUM (over FLOAT)	[0-9]→G, .→H', [eE]→I
H, H'	{14, 15}, {13}	FLOAT	[0-9]→itself, [eE]→I
I	{16, 17}	—	[+~]→J, [0-9]→K
J	{17}	—	[0-9]→K
K	{17, 18}	FLOAT	[0-9]→K

The construction gives 12 states. The slide draws 11, because H and H' are equivalent and it has already merged them. The two labelling decisions are C (IF beats ID) and G (NUM beats FLOAT).

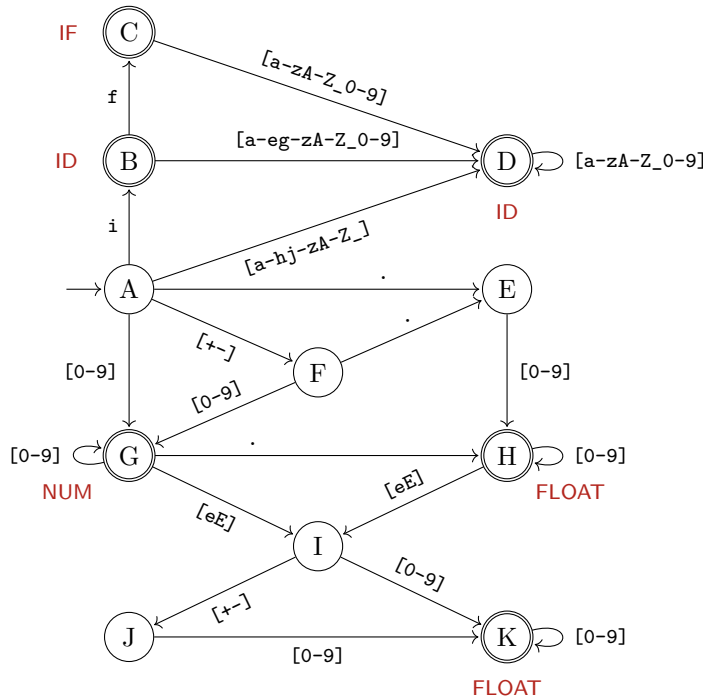


Figure 2: The combined DFA (slide 80). Missing transitions go to the dead state.

Worked example: Slide 80's exercise: `if 17` and `3e-y`

Assume a blank-skipping rule and a catch-all, neither of which is drawn. A read counts every character fetched, including the one that gets the DFA stuck.

Reads	Path	Result
i, f, $\square$	$A \rightarrow B \rightarrow C$ (IF) $\rightarrow$ stuck	emit IF , push back $\square$
$\square$, 1	blank rule, then stuck	skip the blank, push back 1
1, 7	$A \rightarrow G \rightarrow G$ (NUM), end of input	emit NUM 17
3, e, -, y	$A \rightarrow G$ (NUM) $\rightarrow I \rightarrow J \rightarrow$ stuck	emit NUM 3 , push back e-y
e, -	$A \rightarrow D$ (ID) $\rightarrow$ stuck	emit ID e , push back -
-, y	$A \rightarrow F \rightarrow$ stuck, never accepting	catch-all: error on -
y	$A \rightarrow D$, end of input	emit ID y

That is 7 reads for **if 17** and 9 reads for **3e-y**. After **3e-** the DFA hoped for an exponent, and it had to give back three characters.

Technique: How far can a scanner push back?

The maximum pushback is $1 + \ell$, where ℓ is the length of the longest path through non-accepting DFA states that starts right after an accepting state. Here that path is $G \rightarrow I \rightarrow J$, so the maximum pushback is $2 + 1 = 3$. A reachable *cycle* of non-accepting states means the pushback is unbounded, and that is the source of the quadratic trap in Section 9.

5.2 Speed versus memory, and minimisation

The number of DFA states can be **exponential** in the size of the regular expression. Take $L_k = (a \mid b)^* a (a \mid b)^{k-1}$, the strings whose k -th symbol from the end is a . An NFA needs $k + 1$ states, but every DFA needs at least 2^k .

To see why, let $u \neq v$ be strings of length k that differ at position i , with $u_i = a$ and $v_i = b$. The k -th symbol from the end of ub^{i-1} is $u_i = a$, so that string is in L_k , while vb^{i-1} is not. A DFA in the same state after u and after v would treat both extensions alike, so all 2^k strings need different states. So the choice is between low-memory, slower NFAs and high-memory, fast DFAs. Lexer patterns rarely blow up: for `count.1` (Section 8), `flex -v` reports 15 NFA states and 7 DFA states. Tools like `grep` build DFA states lazily, only when the input reaches them.

Every regular language has a **unique minimal DFA**. Chapter 3's table filling finds it in $O(n^2)$. **Hopcroft's algorithm** finds it in $O(n \log n)$ by partition refinement. It starts from the blocks $\{F, Q \setminus F\}$ and splits blocks whose states disagree on which block a letter leads to. When a block splits, only the *smaller* half is used for further splitting, so each state is charged at most $\log_2 n$ times. On the slide-76 DFA for $(a \mid b)^* ac$, the splitter $(\{s'_3\}, c)$ separates s'_1 from $\{s'_0, s'_2\}$, and nothing else splits: 3 states. Minimisation cannot remove a *necessary* blow-up, though: L_k 's minimal DFA still has 2^k states.

The full pipeline (slide 82) is: token patterns, then regular expressions, then Thompson's construction to an NFA, determinisation to a DFA, minimisation, and the analyser. The **Kleene construction** turns a DFA back into a

regular expression, which proves that all three formalisms describe the same languages.

6 When the Lexical Structure Fights Back

So far, the next token depended only on the characters, and the lexer needed only one DFA state of memory. Slide 83 shows three languages that break this.

Fortran: blanks do not matter. `D0 5 I = 1,25` is a loop, but `D05I = 1.25` assigns to the variable `D05I`. Without blanks, both lines start `D05I=1`, and they differ only at the comma versus the point. Longest match alone would read `D05I` as an identifier both times. The lexer needs **lookahead beyond the token**. In `flex` this is trailing context, `r/s`, which matches `r` only if `s` follows and does not consume `s`:

```
D0/[A-Z0-9]*=[A-Z0-9]*, printf("[KEYWORD D0]");
```

With an identifier rule after it, `D05I=1,25` gives `[KEYWORD D0] [NUM 5] [ID I]...`, while `D05I=1.25` gives `[ID D05I]...` (real output). The keyword rule wins because `flex` counts the trailing context in the match length.

PL/I: keywords are not reserved. In `IF THEN THEN THEN = ELSE; ELSE ELSE = IF`, the keywords are the 1st `IF`, the 2nd `THEN` and the 1st `ELSE`; every other word is a variable. Only the parser knows a word's role, so the lexer either returns a generic `IDENT`, or the parser tells it what to expect (*feedback*). `C` has a milder form of the problem: in `T * x;`, `T` may be a typedef name, which makes the line a declaration, or a variable, which makes it a multiplication. `C` compilers let the lexer consult the symbol table, a trick known as the “lexer hack”.

Python: indentation is syntax. Blocks are marked only by indentation, as in the slide's example:

```
if w == z:
    a = b
else:
    e = f
g = h
```

The lexer records the current indentation and invents `INDENT` and `DEDENT` tokens, which act like braces. Python's own tokenizer produces:

```
if w == z : NEWLINE INDENT a = b NEWLINE DEDENT else : NEWLINE
INDENT e = f NEWLINE DEDENT g = h NEWLINE ENDMARKER
```

Mechanism: The indentation stack

The stack starts as `[0]`; blank and comment-only lines are skipped. For a line with `w` leading spaces:

- if $w > top$, push w and emit `INDENT`;
- if $w < top$, pop and emit `DEDENT` until $top \leq w$, and report an error if $top \neq w$;
- if $w = top$, emit nothing.

At end of file, emit one `DEDENT` per entry above 0. For example, indents 0, 4, 8, 0 give `INDENT`, `INDENT`, then `DEDENT` `DEDENT` on the last line. Each `DEDENT` pops something an earlier `INDENT` pushed, so there are at most as many `DEDENTs` as lines: $O(1)$ amortised per line.

Proposition 6.1 (No finite automaton can lex indentation). *No finite-state machine that reads the source and outputs tokens produces the correct `INDENT` and `DEDENT` tokens for every file.*

Proof. Suppose a machine with N states could. Let p_d be the d lines “ j spaces, then x :” for $j = 1, \dots, d$. After p_d the stack is $[0, 1, \dots, d]$, and a final line y with no indentation must produce exactly d `DEDENTs`. Among $p_1, \dots, p_{N+1}$, two prefixes p_i and p_j with $i < j$ end in the same state (pigeonhole). On the same suffix y , the machine then produces the same output after both, but the correct outputs contain $i \neq j$ `DEDENTs`. $\square$

This is the balanced-parentheses argument again: the lexer needs a DFA *plus* a stack.

7 Writing a Lexer by Hand

You can write an ad-hoc scanner, or generate one with `flex`. The slide calls the first more efficient but tedious, and the second less efficient but portable. Production compilers that hand-write their lexers (Clang, Go, Rust) do it mainly for control over error messages and context tricks. The scanner below is from the Stanford Dragon-book notes (slides 85–90).

```
#define T_SEMICOLON ';' // single-char tokens use their ASCII code
#define T_LPAREN '('
#define T_WHILE 257 // everything else starts above 255
#define T_IDENTIFIER 268
#define T_INTEGER 269
#define T_END 349 // end of file
#define T_UNKNOWN 350 // unrecognised character (the catch-all)
struct token_t {
    int type; // the token name
    union { // the attribute, one of:
        char stringValue[256]; int intValue; double doubleValue;
    } val;
};
```

A one-character token is its own ASCII code, so the scanner can return the character it read. Other tokens start at 257, above all 8-bit codes, which is also the yacc convention. The `union` holds one attribute at a time, and `type` says which one is valid. `InitScanner` fills a table of reserved words, so

keywords are found by *lookup*. The driver on slide 86 is just a loop, while (ScanOneToken(stdin, &token) != T_END), which processes one token per iteration. In a compiler, the parser makes these calls.

```

1 static int ScanOneToken(FILE *fp, struct token_t *token) {
2     int i, ch, nextch;
3     ch = getc(fp);
4     while (isspace(ch)) ch = getc(fp);    // discard white space
5     switch (ch) {
6         case '/':                        // comment or T_DIVIDE?
7             nextch = getc(fp);
8             if (nextch == '/' || nextch == '*')
9                 ;                        // "skip the comment here"
10            else
11                ungetc(nextch, fp);        // then fall through
12            case ';': case ',': case '=':  // ... single-char tokens
13                token->type = ch; return ch;
14            case 'A': case 'B': case 'C':  // ... uppercase: keywords
15                token->val.stringValue[0] = ch;
16                for (i = 1; isupper(ch = getc(fp)); i++)
17                    token->val.stringValue[i] = ch;
18                ungetc(ch, fp); token->val.stringValue[i] = '\0';
19                token->type = lookup_reserved(token->val.stringValue);
20                return token->type;
21            case 'a': case 'b': case 'c':  // ... lowercase: identifiers
22                token->type = T_IDENTIFIER; token->val.stringValue[0] = ch;
23                for (i = 1; islower(ch = getc(fp)); i++)
24                    token->val.stringValue[i] = ch;
25                ungetc(ch, fp); token->val.stringValue[i] = '\0';
26                if (lookup_syntab(token->val.stringValue) == NULL)
27                    add_syntab(token->val.stringValue);    // symbol table
28                return T_IDENTIFIER;
29            case '0': case '1': case '2':  // ... digits
30                token->type = T_INTEGER; token->val.intValue = ch - '0';
31                while (isdigit(ch = getc(fp)))
32                    token->val.intValue = token->val.intValue * 10 + ch - '0';
33                ungetc(ch, fp); return T_INTEGER;
34            case EOF: return T_END;
35            default:                        // the catch-all
36                token->val.intValue = ch; token->type = T_UNKNOWN;
37                return T_UNKNOWN;
38    }
39 }

```

Scanner concept	In this code
DFA start state and its transitions	switch (ch), one case each
self-loop, longest match	loops on isupper, islower, isdigit
push back the stuck character	ungetc(ch, fp)
token label	the returned code
catch-all rule	default: returning T_UNKNOWN

The **switch** works because each token class can be recognised by its first character. Each loop reads one character too many and pushes it back. Digits are converted on the fly: 407 becomes 4, then 40, then 407.

Debugging: What is wrong with the slide's scanner

Assembled and run on `WHILE (count) x = x / 2; // halve` followed by `IF (x) RETURN y12; WHILEX`, it prints (real output):

```
T_WHILE '(' ID(count) ') ' ID(x) '=' ID(x) '/' INT(2) ';' '/' ID(halve)
T_IF '(' ID(x) ') ' T_RETURN ID(y) INT(12) ';' T_UNKNOWN
```

1. **Comments are not skipped.** The empty statement after `if` falls through and returns `'/'`, and then the comment text is scanned as code. The fix is to skip to the end of the comment and then `return ScanOneToken(fp, token);`.
2. **Identifiers are lowercase only**, so `y12` becomes two tokens. Upper-case words that are not keywords, like `WHILEX`, become `T_UNKNOWN`.
3. **No bounds checks.** An identifier over 255 characters overflows `stringValue`, and long digit strings overflow `intValue`.
4. **`ungetc` guarantees only one character of pushback.** That is enough here, but not for floats like `3e-y`, which need three. Such scanners keep their own input buffer.

Two-character operators use the same peek-then-decide pattern:

```
case '<': nextch = getc(fp);
        if (nextch == '=') return T_LE;    // commit: "<="
        ungetc(nextch, fp); return '<';    // push back: just "<"
```

8 Generating a Lexer with flex

`flex` implements Chapter 3 and this chapter end to end. It reads the regular expressions, builds the combined NFA, converts it to a DFA, minimises it, and writes C code (`lex.yy.c`, function `yylex`). Conflicts are resolved by longest match, then rule order. `flex -v` prints the statistics, for example “15/2000 NFA states, 7/1000 DFA states” for `count.1` below.

```
%{
Declarations          /* C code copied verbatim to the top: includes, globals
                        */
%}
Definitions           /* options and named patterns, used later as {name} */
%%
Rules                  /* pattern  action */
%%
User subroutines       /* C code copied verbatim to the end, often main() */
```

How a rule is chosen. Each call to `yylex()` goes through these steps:

1. find the longest prefix that *any* rule matches;
2. break ties in favour of the rule written **first**;
3. set `yytext` and `yylen` and run the action;
4. if the action executes `return`, `yylex` returns that value to the parser; otherwise it keeps scanning.

If no rule matches a character, the hidden **default rule** copies it to the output. Use `%option nodefault` or your own catch-all to prevent this.

Syntax	Meaning	Example
.	any character except newline	"#".*
[^x]	complement class, newline included	[^0-9]
x{n,m}	<i>n</i> to <i>m</i> repetitions	[0-9]{1,3}
"x", \x	literal text, escaped character	"/*", \.
{name}	a named pattern from the definitions	{digit}+
r/s	<i>r</i> if followed by <i>s</i>	the Fortran rule
<SC>r	<i>r</i> only in start condition SC	<COMMENT>.

Patterns must start in column 1, and an unquoted space ends a pattern (write " " or []).

Interface / API: Talking to the scanner

`yylex()` scans `yyin` (`stdin` by default). `yytext` and `yylen` hold the lexeme and its length. `yyval` holds the token's attribute, and `yyloc` its line and column; both are normally declared by `bison`. `ECHO` copies the lexeme to the output. `yywrap()` returning 1 means "no more input". Link with `-ll` on macOS or `-lfl` on Linux (the slides' "ll" lost its dash), or write `%option noyywrap` and your own `main`.

Worked example: Examples 1 and 2: hiding digits, counting like `wc`

```
%%
[0-9]+ printf("?");
.      ECHO;
```

On call `0612345678` at 9am it prints `call ? at ?am`: longest match swallows each run of digits whole. No rule matches the newline, because `.` excludes it, so the default rule echoes it; `flex -d` even prints "accepting default rule".

```
\n      {numLines++; numChars++;}
[^\t\n]+ {numWords++; numChars += yylen;}
.       {numChars++;}
```

On `hi there` followed by `yo` on a second line, it counts 12 characters, 3 words and 2 lines, like `wc`. Rule order matters here. On a one-letter word, the word rule and `.` tie at length 1, and the earlier rule wins. With the last two rules swapped, `a b cd` counts as 1 word instead of 3 (checked).

Worked example: Example 3: a typical compiler front end (slides 98–99)

```

delim    [ \t\n]
ws       {delim}+
letter   [A-Za-z]
digit    [0-9]
id       {letter}({letter}|{digit})*
number   {digit}+(\.{digit}+)?(E[+-]?{digit}+)?
%%
{ws}      { /* no action and no return */ }
if        {return(IF);}
then      {return(THEN);}
else      {return(ELSE);}
{id}      {yyval = (int) installID(); return(ID);}
{number}  {yyval = (int) installNum(); return(NUMBER);}
"<"      {yyval = LT; return(RELOP);}
"<="     {yyval = LE; return(RELOP);}
...       /* likewise "=", "<>", ">", ">=" */

```

- Blanks are matched and dropped (job 4).
- The keyword rules come before {id}, so if is IF by priority, while ifx is ID by length.
- installID enters yytext into the symbol table and passes the entry through yyval (job 3).
- The relational operators share RELOP, with the attribute telling them apart.

Run with a small driver (real output, wrapped):

```

if x1 <= 3.14E-2 then y else ifx <> x1
<IF> <ID,0:x1> <RELOP,LE> <NUMBER,0:3.14E-2> <THEN> <ID,1:y> <ELSE> <ID
,2:ifx>
<RELOP,NE> <ID,0:x1>

```

Both occurrences of x1 get symbol-table index 0. On d=2.e5, the result is NUMBER 2, then a stray . echoed by the default rule, then ID e5. A real compiler needs a catch-all to report that point. Finally, storing a pointer in an int breaks on 64-bit machines; bison's %union fixes that.

Line and column. flex runs the macro YY_USER_ACTION before *every* action, including the empty actions of skipped comments. That makes it the place to keep positions:

```

%{
int line = 1, col = 1, tline, tcol; /* next char / current lexeme */
#define YY_USER_ACTION tline = line; tcol = col; \
for (int i = 0; i < yytext[i]; i++) \
if (yytext[i] == '\n') { line++; col = 1; } else col++;
#define OUT(k) printf("%d:%d %s %s\n", tline, tcol, k, yytext)
%}
%%
"(*([~*]|"~*+([~*]))"~*+)" { /* block comment, may span lines */ }
let|print OUT("KEYWORD");
0|[1-9][0-9]* OUT("INT");
. OUT("error: unexpected");

```

In a full version of this scanner (real run), a `print` that follows a two-line comment is correctly reported at 3:20. The input `02` gives `INT 0` then `INT 2`, because the integer rule forbids leading zeros.

Start conditions give the scanner modes, which is often cleaner than one huge pattern (`depth` is an `int` declared in the declarations part):

```
%x COMMENT
%%
"(" { depth = 1; BEGIN(COMMENT); }
<COMMENT>"(" { depth++; }
<COMMENT>"(" { if (--depth == 0) BEGIN(INITIAL); }
<COMMENT>.\n { }
<COMMENT><EOF> { printf("[error: unterminated comment]"); return 0; }
```

With an identifier rule added, it was run on one line (real output):

```
input:  x (* a (* nested *) still *) y (* open
output: [ID:x][ID:y][error: unterminated comment]
```

The depth counter handles *nested* comments, which no regular expression can.

Debugging: Debugging flex

- `flex -d` traces every accepted rule.
- “**warning, rule cannot be matched**” means an earlier rule always matches the same text at least as long.
- Characters that no rule prints come from the default rule.
- An indented pattern is treated as C code.

Worked example: Predict the output

```
a+b    printf("[X:%s]", yytext);
ab*    printf("[Y:%s]", yytext);
b      printf("[Z]");
ab     printf("[W]");
.\n    ECHO;
```

The input is `aaabbab`, a newline, `abbbca`, a newline. At each position, take the longest match; on a tie, the earlier rule wins.

- `aaab`: `a+b` matches 4 characters against 1 for `ab*`, so X.
- `b`: Z.
- `ab`: X, `ab*` and `ab` all match 2, so X wins as the first rule.
- `abbb`: `ab*` matches 4, so Y.
- `c` is echoed.
- `a`: Y.

Real output:

```
predict.1:6: warning, rule cannot be matched
[X:aaab][Z][X:ab]
[Y:abbb]c[Y:a]
```

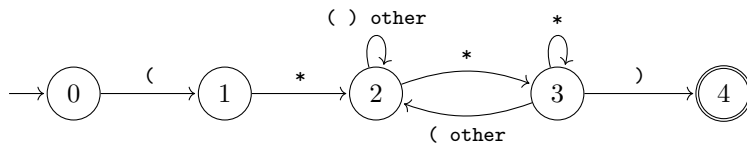
Rule `ab` never fires, because `a+b` matches the same text and comes first.

9 Tricks, Techniques and Classic Problems

Technique: Exam-speed recipes

- **Thompson counts (merged):** $|Q| = 2(\text{leaves} + \#| + \#*) - \#\text{concat}$ and $|E| = \text{leaves} + 4(\#| + \#*)$. Parse first.
- **NFA by hand:** write S as a sorted set after every character, and take the closure after every move.
- **Longest match by hand:** keep the label and end of the last accepting step. When stuck, emit it, push back what follows, and restart. Rule order only breaks equal lengths.
- **Counting reads:** count every character fetched, including the stuck one, and count re-reads again. State whether end of file counts.
- **Not regular, or needs many states:** find prefixes that must lead to different states. Separate them with a suffix, or use pigeonhole.

Deriving a comment pattern from its DFA. The closer must be the *first* one after the opener, so do not guess: draw the DFA. For $(* \dots *)$, over the classes $(, *,)$ and *other*, where *other* includes newline:



State 2 means “inside, last character not $*$ ”. State 3 means “inside, just read $*$ ”. With a dead state, the DFA has 6 states, and it is minimal. Inside the comment, the machine either stays in state 2 on $[^*]$, or makes the round trip $2 \rightarrow 3 \rightarrow 2$ on $"*" + [^*]$. It leaves on $"*" + ")"$. Hence

$$"(*"([^\ast] | "*" + [^\ast]))"*" + ")"$$
 and, for C comments,

$$"/*"([^\ast] | "*" + [^\ast /])"*" + "/"$$

flex confirms the C version (real runs):

- `/**/, /****/` and `/* a * b **/` are whole comments.
- `/*/` is not a comment.
- `/* a */ b */` matches only `/* a */`.

Warning / Gotcha: The tempting rule `"(*".*")"`

On `x (* c1 *) y (* c2 *) z` it matches `( * c1 *) y ( * c2 *)`. Longest match pushes `.*` to the *last* closer on the line, and `y` vanishes (real output). Since `.` cannot cross a newline, a two-line comment is not matched at all, and its delimiters leak out as junk. Changing rule order cannot help: the problem is length, not a tie.

The quadratic trap. Take rules $X = ab$ and $Y = (ab)^*c$ on $(ab)^k$ with no c . Every token reads to the end of the input hoping for a c , and then falls back to ab . Token j costs $2(k - j + 1)$ reads, so the total is

$$\sum_{j=1}^k 2(k - j + 1) = k(k + 1) = \Theta(n^2),$$

and one token can push back $n - 2$ characters. Reps' fix (Chapter 3) memoises the hopeless (state, position) pairs. Every later token then stops after 3 reads, for $5k - 4$ reads in total. Both counts were checked by simulation.

Worked example: Short classic problems

P1. The merged Thompson NFA for $(ab \mid c)^*$ has $2(3 + 1 + 1) - 1 = 9$ states and $3 + 8 = 11$ edges. Chapter 3's convention gives 10 states and 12 edges.

P2. With rules $[a-z]^+$ (ID) before "if" (IF), the input `if` gives ID: equal length, so the earlier rule wins, and `flex` warns that IF cannot be matched.

P3. With rules `INT 0|[1-9][0-9]*` and `ID [a-z]^+`, `0x1F` gives INT 0, ID `x`, INT 1, then an unmatched `F`. A HEX rule `0[xX][0-9a-fA-F]^+` wins by length, wherever it is placed.

P4. With `NUM [0-9]^+`, `FLT [0-9]^+.[0-9]^+` and `DOT`, the maximum pushback is 2, for example on `3.x`.

P5. Indents 0, 4, 8, 2: the last line pops 8 and 4, then $0 \neq 2$ gives an inconsistent dedent.

Problem Set 4	Prepared in
1. Comment regex, greedy rule, minimal DFA, numbers	comment DFA and greedy rule (§9); float pattern and sign trap (§2); leading zeros (§8)
2. Thompson size, structure and cost	Propositions 3.1, 3.2 and 3.3 (§3)
3. Maximal munch, pushback, quadratic trap	traces in §4 and §5; pushback bound; quadratic trap (§9)
4. Exponential DFAs	§5.2; subset construction (§5)
5. Indentation, Fortran	§6
6. <code>flex</code> : predict and write	§8: rule choice, predict example, <code>YY_USER_ACTION</code> , comments

Notes on the original slides

- **Slide 63.** The 8-state NFA shown is not built by the Thompson rules of slide 62, and it breaks property 2 printed on the same slide. Thompson's rules give 10 states.
- **Slide 65.** "The first one to reach an accepting state" must be read with longest match (slide 68), or `ifu26` would be cut after one character.
- **Slide 73.** $O(|S||Q|^2)$ holds for arbitrary NFAs. Thompson NFAs need only $O(|S||Q|)$.

- **Slide 80.** The drawn DFA has already merged two equivalent states (12 become 11). Its exercise assumes a blank rule and a catch-all that are not drawn.
- **Slide 88.** The comment branch falls through and returns `'/'`.
- **Slides 96–98.** `11` should be `-11` or `-1f1`, and `(int) installID()` truncates pointers on 64-bit machines.

Summary: Chapter 4 in eight lines

1. The lexer cuts, labels, records names and drops blanks; the parser pulls one token $\langle \text{name}, \text{attribute} \rangle$ at a time.
2. Give tokens different names only where the grammar must tell them apart.
3. Dragon-book Thompson merges states on concatenation: $|Q| = 2(\text{leaves} + \#| + \#*) - \#\text{concat}$, with one entry, one exit and at most two out-edges per state.
4. NFA simulation is $S \leftarrow \varepsilon\text{-closure}(\text{move}(S, c))$. It costs $O(|S||Q|^2)$ in general, $O(|S||Q|)$ for Thompson NFAs and $O(|S|)$ for a DFA.
5. Many tokens: merge, take the longest match, push back, break ties by priority, catch the rest.
6. DFA states carry the highest-priority label. DFAs can be exponential, and minimisation (Hopcroft) cannot cure a necessary blow-up.
7. Fortran needs lookahead, PL/I needs the parser, and Python needs a stack.
8. A hand-written scanner is a DFA in disguise. `flex` applies longest match, then rule order, plus a default rule that echoes.